

CHAPTER SIX

PL/SQL and Database Administrator Tasks

Learning Objectives

- After completing this chapter, the learner should be familiar with the following concepts in PL/SQL.
 - Structure of PL/SQL, PL/SQL elements and steps to create PL/SQL program
 - Basic concepts of Procedure and functions
 - Basic concept of Trigger, types of trigger, DDL trigger, DML Trigger
 - Database startup and shutdown
 - Database backup and recovery
 - Auditing database activities

6.1. Shortcomings in SQL

- SQL is a powerful tool for accessing the database but it has the following **limitations**:
 1. SQL statements can be executed only one at a time.
 2. Database generates its own error message, which is sometimes difficult to understand.
 3. SQL is not able to do the conditional query on RDBMS.

6.2. PL/SQL

- PL/SQL stands for Procedural Language/Structured Query Language
- It is provided by Oracle as a procedural extension to SQL.
- SQL is a declarative language.
- In SQL, the statements have no control to the program and can be executed in any order.
- PL/SQL is a procedural language that makes up for all the missing elements in SQL.
- PL/SQL allows users and designers to develop complex database applications.
- Like SQL, PL/SQL is not case sensitive.

6.1. PL/SQL(2)

- The major goals of PL/SQL is to:
 - Increase the expressiveness of SQL,
 - Process query results in a tuple-oriented way,
 - Optimize combined SQL statements,
 - Develop modular database application programs,
 - Reuse program code, and
 - Reduce the cost for maintaining and changing applications.

6.3. Structure of PL/SQL

- PL/SQL is a 4GL (fourth generation) programming language.
- PL/SQL is a block structured language.
- PL/SQL program is made up of blocks, where block is a smallest piece of PL/SQL code having logically related statements and declarations.
- A block consists of **three sections namely:**
- **Declare Section:** declares the variables, constants, processes, functions, etc., to be used in the other parts of program.
- **Begin Section:** A set of SQL and PL/SQL statements, which is executed when PL/SQL block runs.
- **Exception Section:** Handles the errors, which occurs during execution of the PL/SQL block.
- **End Section:** indicates the end of PL/SQL block.
- Every PL/SQL program must consist of at least one block.

6.4. Procedure

- A procedure is a subprogram that performs some specific task, and stored in the data dictionary.
- Stored procedures are PL/SQL programs to define a business process.
- When a procedure is created, the Oracle automatically performs the following steps:
 1. Compiles the procedure
 2. Stores the procedure in the data dictionary
- **System Privileges for Procedures:** CREATE PROCEDURE, CREATE ANY PROCEDURE, EXECUTE ANY PROCEDURE...

The syntax for creating a procedure is follows

- **CREATE OR REPLACE PROCEDURE** [schema.] package name [(argument {IN, OUT, IN OUT} data type,)] {IS, AS} [local variable declarations]

- **BEGIN**

executable statements

- **EXCEPTION**

exception handlers

- **END** [procedure name];

- **Create:** Creates a new procedure, if a procedure of same name already exists, it gives an error.
- **Replace:** Creates a procedure, if a procedure of same name already exists, it replaces the older one by the new procedure definition.
- **Schema:** If the schema is not specified then procedure is created in user's current schema.

The syntax for creating a procedure is follows

- **Argument:** It is the name of the argument to the procedure.
- **IN:** Specifies that a value for the argument must be specified when calling the procedure.
- **OUT:** Specifies that the procedure pass a value for this argument back to its calling environment after execution.

Executing/Invoking a Procedure

- In SQLPLUS it can be execute using the EXECUTE command, followed by the procedure name.

Removing a Procedure

- To remove a procedure, one must have DROP ANY PROCEDURE privilege.
- To remove a procedure completely from the database, the command is:
- *DROP PROCEDURE <PROCEDURE NAME>;*

6.5. Function

- A Function is similar to procedure except that it must return one and only one value to the calling program.
- Function can be used as part of SQL expression, whereas the procedure cannot.
- Stored functions are PL/SQL programs that can be used to create user-defined functions to return a value.

The Difference Between Function and Procedure

- A procedure never returns a value to the calling portion of code, whereas a function returns exactly one value to the calling program.
- It is mandatory for a function to have at least one RETURN statement, whereas for procedures there is no restriction.
- A procedure may have a RETURN statement or may not.

The exact syntax for defining a function

- CREATE OR REPLACE FUNCTION [schema.] functionname [(argument IN datatype,)] RETURN datatype {IS,AS} [local variable declarations];
- BEGIN //executable statements;
- EXCEPTION //exception handlers;
- END [functionname];

Purity of a Function

- Function must satisfy the following requirements, which are known as **Purity Rules**:
- When called from a SELECT statement or a parallelized INSERT, UPDATE, or DELETE statement, the function cannot modify any database tables.

Purity of a Function

- When called from an INSERT, UPDATE, or DELETE statement, the function cannot query or modify any database tables modified by that statement.
- When called from a SELECT, INSERT, UPDATE, or DELETE statement:
- The function cannot execute SQL transaction control statements, session control statements (such as SET ROLE), or system control statements (such as ALTER SYSTEM).
- Also, it cannot execute DDL statements (such as CREATE) because they are followed by an automatic commit.
- If any of the above rules is violated, the function is said to be **not following the Purity Rules**.
- **Removing a Function:** *DROP FUNCTION <FUNCTION NAME>;*

6.6. Database Triggers

- A trigger is a PL/SQL program unit that is executed when an event occurs.

Privileges Required for Triggers

- Create or alter a TRIGGER on a specific table requires TRIGGER privileges table privileges:
- To create TRIGGER in one's own schema must have CREATE TRIGGER privilege.
- To create a trigger in any other's schema must have CREATE ANY TRIGGER system privilege.
- To create a trigger on table, one must own the table or should have ALTER privilege for that table or should have ALTER ANY TABLE privilege.
- To ALTER a trigger, one must own that trigger or should have ALTER ANY TRIGGER privilege.
- To create a TRIGGER on any database level event, one must have ADMINISTER DATABASE TRIGGER system privilege.

Create Trigger Syntax

- CREATE [OR REPLACE] TRIGGER <trigger name>
- [BEFORE/AFTER/INSTEAD OF]
- [INSERT/UPDATE/DELETE [of column,..]] ON <table name>
- [REFERENCING [OLD [AS] <old name> | NEW [AS] <new name>]
- [FOR EACH STATEMENT/FOR EACH ROW]
- [WHEN <condition>]
- [BEGIN
- PL/SQL block
- END];

Dropping Triggers

- User needs to have DROP ANY TRIGGER system privilege to drop a trigger.
- Triggers can be dropped like tables using the drop trigger command.
- The drop trigger command removes the trigger structure from the database.
- **Syntax:** *DROP TRIGGER <trigger name>;*

Types of trigger

- **BEFORE Triggers:** it execute the trigger action before the triggering statement is executed.
- It is used to derive specific column values before completing a triggering DML, DDL statement.
- **AFTER Triggers:** it execute the trigger action after the triggering statement is executed.
- **INSTEAD-OF Triggers:** used to tell Oracle what to do instead of performing the actions that executed the trigger.
- **Enabling and Disabling a Trigger:** By default, a trigger is enabled when it is created.
- **Enabled:** it executes its trigger action if a triggering statement is issued.
- **Disabled:** It does not execute its trigger action, even if a triggering statement is issued.
- **Syntax:** *ALTER TRIGGER <Trigger name> ENABLE/DISABLE;*

Types of Triggers

- **Triggers on System events:** The trigger fire when instance startup and shutdown and error messages.
- **Trigger on User events: LOGON** triggers fire after a successful logon of a user.
- **LOGOFF:** triggers fire at the start of a user logoff.
- **Row Level Triggers:** it is fired for each row that will be affected by the SQL statement, which fires the trigger. E.g. Update
- **Statement Level Triggers:** is fired only once on behalf of the triggering SQL statement. E.g. if a DELETE statement deletes several rows from a table.
- **Triggers on DDL Statements:** This trigger gets fired when DDL statement such as CREATE, ALTER, or DROP command is issued.
- DDL triggers can be associated with the database or with a schema.

6.7. Starting Up and Shutting Down an Oracle Instance

- As a DBA, you are responsible for the startup and shutdown of the Oracle instance.
- To start up or shut down an Oracle instance needs appropriate privileges such as **SYSDBA** and **SYSOPER**.
- The **SYSDBA** authorization is an all-empowering authorization that allows you to perform **any database task**.
- The **SYSOPER** authorization is a less powerful authorization that allows startup and shutdown abilities.

6.7.1. Starting Up an Oracle Database Instance

- Oracle instance is composed of a set of logical memory structures and background processes.
- When Oracle is started, these memory structures and background processes are initialized and started so that users can communicate with the Oracle database.
- When database starts up, a database passes through **three modes: NOMOUNT, MOUNT, and OPEN**.

6.7.1. Starting Up an Oracle Database Instance

STARTUP NOMOUNT: This starts the instance **without mounting the** database.

- In this mode, the parameter file is read, and the background processes and memory structures are initiated, but they are not attached & communicate with the disk structures of the database.
- When the instance is in this state, the database is not available for use.
- In NOMOUNT mode, the background processes are started and shared memory is allocated.
- The instance is not associated with any database.
- This state is used to create a database or to create a database control file.
- SQLPLUS / AS SYSDBA
- SQL>STARTUP NOMOUNT;

6.7.1. Starting Up an Oracle Database Instance(2)

STARTUP MOUNT: This performs all the work of the STARTUP NOMOUNT option and **reads the control file.**

- At this point, Oracle obtains information from the control files.
- **In this mode:**
 - Renaming data files,
 - Enabling or disabling archive logging,
 - Renaming and adding redo log files and
 - Recovering the database are performed.
- SQLPLUS / AS SYSDBA
- SQL>Startup Mount; or
- SQL>Alter database MOUNT;

6.7.1. Starting Up an Oracle Database Instance(3)

STARTUP OPEN: If no mode is specified, it is the default startup mode

- It performs all the steps of the STARTUP NOMOUNT and STARTUP MOUNT options.
- This option makes the database available to all users.
- STARTUP OPEN READ ONLY opens the database in read-only mode.
- STARTUP OPEN RECOVER opens the database and performs a database recovery.
- SQLPLUS / AS SYSDBA
- SQL>Startup OPEN; or
- SQL>Alter database OPEN;

6.7.1. Starting Up an Oracle Database Instance(4)

- **STARTUP FORCE:** You can use this option if you are experiencing difficulty starting the database in a normal fashion.
 - E.g. when a database server loses power and the database stops abruptly;
 - It can be used if a normal startup does not work.
 - STARTUP FORCE does a shutdown abort and then restarts the database;
 - It can be used to cycle a database that is in open state.
- **STARTUP RESTRICT:** It starts up the database and places it in OPEN mode.
 - But gives access only to users who have the RESTRICTED SESSION privilege.
- **STARTUP UPGRADE:** This option starts up the database in upgrade mode.
- **STARTUP DOWNGRADE:** This option enables database downgrade scripts to be run.
- `SQL>STARTUP [NOMOUNT|MOUNT|OPEN] [PFILE=] [RESTRICT] [FORCE] [QUIET]`

6.7.2. Shutting Down an Oracle Database Instance

- **SHUTDOWN NORMAL:** If no shutdown options is specified, It is default shutdown type.
 - No new Oracle connections are allowed.
 - The database will wait until all users are disconnected.
 - Also known as a clean shutdown because when you start Oracle again no recovery is necessary.
- **SHUTDOWN TRANSACTIONAL:** It is a bit more aggressive than a normal shutdown.
 - No new Oracle connections are allowed.
 - No new transactions are allowed.
 - Once all active transactions on the database have completed, all client connections are disconnected

6.7.2. Shutting Down an Oracle Database Instance(3)

- **SHUTDOWN IMMEDIATE:** It is the next most aggressive option.
 - No new Oracle connections are allowed.
 - Any uncommitted transactions are rolled back.
 - So, user in the middle of a transaction will lose all the uncommitted work.
 - Oracle does not wait for clients to disconnect.
 - User database connections are terminated.
- **SHUTDOWN ABORT:** It is the most aggressive type of shutdown.
 - No new Oracle connections are allowed.
 - Any SQL statements currently in progress are terminated, regardless of their state.
 - Uncommitted work is not rolled back.
 - Oracle disconnects all client connections immediately.
 - Do not use SHUTDOWN ABORT regularly.
- SQLPLUS / AS SYSDBA
- SQL>SHUTDOWN [NORMAL|TRANSACTIONAL|IMMEDIATE|ABORT]

6.8. Backup and Recovery

- As a database administrator, your primary goal is to keep the database open and available for users, usually 24 hours a day, seven days a week.
- Backup and recovery is the most important task of DBA.
- The server's system administrator includes the following tasks:
 - Proactively solving common causes of failures
 - Increasing the mean time between failure (MTBF)
 - Ensuring a high level of hardware redundancy
 - Increasing availability
 - Decreasing the mean time to recover (MTTR)
 - Minimizing or eliminating loss of committed transactions

6.8.1. Backup and Recovery Concepts

- Control file keeps repository information by default.
- Repository is a collection of information about backups.
- **Control file:** it is a small binary file that contains information about the structure of the database. Control file include the following:
 - The database name to which the control file belongs.
 - The database-creation timestamp.
 - The name, location, and online/offline status information of the data files.
 - The name and location of the redo log files.
 - Redo log archive information.
 - Tablespace names.
 - The current log sequence number.
 - The most recent checkpoint information.
 - The beginning and ending of undo segments.
 - Recovery Manager's backup information.

6.8.1. Backup and Recovery Concepts(1)

- **Redo Log Files:** A redo log file **records all changes to the database, before the changes are written to the data files.**
- The database maintains online redo log files to protect against data loss.
- To recover from an instance or a media failure, redo log information is required to roll data files forward to the last committed transaction.

6.8.1. Backup and Recovery Concepts(1)

- **Archived Redo Log File:** It is a **copy of a redo log file before it is overwritten by new redo information**.
- To backup of a data file up to the **latest committed** transaction you should configure the database in ARCHIVELOG mode.
- **ARCHIVELOG mode vs NOARCHIVELOG mode:** With NOARCHIVELOG mode, recovery options are limited.
- ARCHIVELOG mode allows hot backups, recovery until the point of failure and time based recovery.
- To know if we are in archive mode or noarchivemode:
 - Use SQLPLUS connect to database as SYSDBA.
 - SQL>archive log list;
- To enable archive log mode, S
 - QL>shutdown immediate;
 - SQL>startup mount;
 - SQL>Alter database archivelog;
- To switch one redo log to the next: SQL>Alter system switch logfile;

6.8.2. Performing Backups

- Oracle has whole backups and partial backups strategies.
- Backup type can be divided into two general categories: full backups and incremental backups.
- Depend on the database is **open or closed**, backups can be further categorized as consistent and inconsistent backup.
- A **consistent backup** (offline backup) is performed while the database is **not open**.
- An **inconsistent backup**(online backup) is performed while the database is **open** and available to users.
- An **incremental backup** Oracle supports five levels of incremental backups from 0 to 4, 0 and 1 are the most commonly used.
- A **full backup** includes all blocks of every data file backed up in a whole or partial database backup.

6.8.2. Performing Backups

- **Backing up the Control File:**
- **An editable text file; this backup is called a backup to trace.**
- The text backup is created using the statement.
- SQL> alter database backup controlfile to trace;
- The file is created in the trace directory under <ADR_HOME>/trace.
- If you want to name, the control file backup you can do this:
- SQL> alter database backup controlfile to trace as '/tmp/mydbcontrol.txt';
- **A binary backup of the control file.** SQL> alter database backup controlfile to '/ora_backup/11GR11/ctlfile20040911.bkp';
- **An RMAN backup of the control file.** Using RMAN, you can back up the control file using the BACKUP CURRENT CONTROLFILE statement.
- RMAN> BACKUP CURRENT CONTROLFILE;

6.8.3. Performing User-Managed Cold Backups

- **Cold backups** are performed after shutting down the database by using SHUTDOWN IMMEDIATE or SHUTDOWN TRANSACTIONAL statement.

6.8.4. Performing User-Managed Hot Backups

- To perform a **hot backup**, the database must be in ARCHIVELOG mode and opened.
- Example, if you want to back up the USERS tablespace, perform the following:
- RMAN> BUCKUP TABLESPACE users;
- You **cannot** perform incremental backups using user-managed backups.
- You must use RMAN for incremental backups.

6.8.5. Using RMAN to Perform Backups

- RMAN is the primary component of the Oracle database used to perform backup and recovery operations.
- We can use RMAN to back up, whole or partial databases, full or incremental, and consistent or inconsistent.
- RMAN performs **true block level backup**, capability of ignoring empty datafile blocks, encrypted backups and compressed backups.
- To display the RMAN backup settings:
 - RMAN target / nocatalog
 - RMAN> show all;

6.8.6. Creating Full and Incremental Backups

- RMAN> backup incremental level 0 as compressed backupset database; //Level 0 backup
- First alter RMAN>Alter system switch logfile;
- RMAN> backup incremental level 1 as compressed backupset database;
- RMAN> list backup; //To see or view backup the reports.
- RMAN>Backup database; // To do a full database backup:

6.9. Recovering from User Errors

- We must do database recovery in a mount state.
- We cannot do database recovery in a **Nomount and Open state**.
- RMAN>Shutdown immediate; SQL> Startup mount;
- RMAN>Restore datafile 6; The datafile is restored.
- A user's data was inadvertently changed or deleted or a table was dropped.

6.10. Auditing Database Activity

- Auditing involves monitoring and recording specific database activity.
- Audit records can be stored in either of these locations: Database and Operating-system files.
- The default is DB, as in `AUDIT_TRAIL=DB` which tells the database to record audit records in the database.
- To change the location of audit records in the spfile and will be in effect after a database restart.
- `SQL>ALTER SYSTEM SET audit_trail=DB SCOPE=SPFILE;`
- After changing the `audit_trail` parameter, you will need to shut down and start up your database instance.
- **Managing Statement Auditing:** monitoring and recording the execution of specific types of SQL statements.

6.10. Auditing Database Activity(2)

- **Enabling Statement Auditing:** You enable auditing of specific SQL statements with an AUDIT statement.
- E.g. to audit the SQL statements CREATE TABLE, DROP TABLE, and TRUNCATE TABLE, use the TABLE audit option like this:
- SQL>AUDIT table;
- To record audit entries for specific users only, include a BY USER clause in the AUDIT statement.
- Example, to audit CREATE, DROP, and TRUNCATE TABLE statements for user Abebe only, execute the following:
- SQL>AUDIT table BY Abebe;
- To further limit auditing to only these unsuccessful executions, use a WHENEVER clause like this:
- SQL>AUDIT table BY Abebe WHENEVER NOT SUCCESSFUL;
- You can alternatively specify WHENEVER SUCCESSFUL to record only successful statements.

6.10. Auditing Database Activity(3)

- If you do not include a WHENEVER clause, both successful and unsuccessful statements will trigger audit records.
- Specify BY ACCESS or BY SESSION in the AUDIT statement, like this:
- SQL>AUDIT INSERT TABLE BY Abebe BY ACCESS;
- **Querying auditing:** DBA_AUDIT_TRIAL and view that overlays the SYS.ADU\$ system table.
- SQL>Show parameter Audit_trial;
- SQL>Select * from dba_audit_trial;
- SQL>Audit select on Abebe.Staff;
- SQL>Audit select on Abebe.Student whenever succussful;
- SQL>Audit delete on Abebe.Course by session whenever not successful;
- SQL>Audit select, insert, update, delete on Abebe.Course Whenever Successful; //By writing single query.

THANK YOU

THE END